

ADDIS ABABA UNIVERSITY

College of Natural and Computational Science

GizeGo Test Plan Document (TPD)

Group Members

1. Sofanit Mesfin (UGR/2417/15)

2. Samrawit Hailaeyesus(UGR/5530/15)

3. Rediet Yalelet(UGR/7139/15)

4. Tesfa Tilahun(UGR/0647/15)

Date: 02/06/20

Advisor: Mrs.Axumawit T.

Content **page**

1. Introduction **1**

1.1 Purpose **2.**

Features to be Tested/not to be tested **3**

2.1 Features to be Tested 5

2.2 Non-Functional Requirements to be Tested 5

2.3 Features Not to Be Tested 8

3. Pass/Fail Criteria **9**

4. Approach/Strategy **10**

4.1 Testing Tools 11

4.2 Training Requirements 11

4.3 Metrics to be Collected 11

4.4 Metric Collection Levels 12

4.5 Configuration Management 12

4.6 Test Configurations 12

4.7 Test Environment 13

4.8 Regression Testing 14

4.9 Handling Untestable or Ambiguous Requirements 14

4.10 Testing Scope and Coverage 14

5. Test cases with specifications **15**

5.1 Functional Test case 15

5.2 Non-Functional Test Cases 25

6. Test Summary Report **26**

6. Conclusion **27**

References **28**

1. Introduction

Software testing is the process of evaluating and verifying a software system to ensure that it functions correctly according to the specified requirements. It is performed to identify errors, defects, or inconsistencies within the system and to ensure that the developed software satisfies user needs and system objectives. Software testing also helps improve software quality, reliability, security, and performance before the system is deployed to end users.[1]

This document presents the Test Plan for the GizeGo Smart Public Transportation and Real-Time Bus Tracking System. The purpose of this testing document is to define the testing scope, objectives, strategies, methods, and test cases used to verify and validate the functionality and performance of the system.

The GizeGo system is a mobile-based application designed to improve public transportation services by providing real-time bus tracking, estimated arrival time (ETA) prediction, route management, GPS simulation, incident reporting, notification services, and multilingual support. The system is developed using React Native for the mobile application, Node.js with Express.js for backend services, Socket.io for real-time communication, and PostgreSQL as the relational database management system.

Testing is conducted to ensure that all functional and non-functional requirements of the system operate correctly according to the software specification. The testing process verifies that users such as passengers, drivers, and administrators can properly interact with the system based on their authorized roles and permissions.

The main objectives of this test plan are:

- To verify that all functional requirements are implemented correctly.
- To ensure the reliability and stability of the system.
- To validate authentication and role-based access control mechanisms.
- To ensure accurate GPS simulation and ETA calculation.
- To verify proper handling of incidents and notifications.
- To identify and correct system errors, defects, and inconsistencies before deployment.

This test plan includes functional testing, integration testing, system testing, security testing, and user acceptance testing for the major subsystems of the GizeGo system.

1.1 Purpose

The purpose of this Test Plan Document is to define the testing procedures, testing scope, testing strategies, and test cases for the GizeGo Smart Public Transportation and Real-Time Bus Tracking System. This document serves as a guideline for verifying and validating that the system satisfies its specified functional and non-functional requirements.

The test plan is intended to ensure that all major components and subsystems of the GizeGo system operate correctly, efficiently, securely, and reliably under different conditions. It also helps identify system defects, inconsistencies, and performance issues before the final deployment of the application.

Specifically, this document aims to:

- Verify the correctness of system functionalities such as route management, GPS simulation, ETA calculation, incident reporting, and notification services.
- Ensure that role-based access control mechanisms function properly for passengers, drivers, and administrators.
- Validate the interaction between the mobile application, backend services, and PostgreSQL database.
- Evaluate the system's performance, reliability, and error-handling capabilities.
- Ensure that the developed system meets user requirements and expected operational behavior.

This test plan provides the foundation for conducting systematic and organized testing activities throughout the development and evaluation phases of the GizeGo system.

2. Features to be Tested/not to be tested

2.1 Features to be Tested

The following major features and functionalities of the GizeGo Smart Public Transportation and Real-Time Bus Tracking System will be tested to ensure that the system operates correctly according to the specified requirements.

1. User Authentication and Account Management

- User registration(for passenger optional for feedback)
- User login and logout
- Password validation
- JWT-based authentication
- Profile viewing and profile update
- Role-based access control for different user types

2. Bus Management

- Add new bus
- Update bus information
- Delete bus records
- View available buses
- Assign bus status

3. Route Management

- Create routes
- Update route information
- Delete routes
- View route details
- Assign buses to routes

4. Stop Management

- Add route stops
- Update stop information
- Delete stops
- View stops by route
- Maintain stop order sequence

5. GPS Simulation and Real-Time Tracking

- Simulate GPS coordinates for buses
- Update bus locations periodically
- Store GPS logs in the database
- Display real-time bus locations on the map
- Real-time location updates using Socket.io

6. ETA (Estimated Time of Arrival) Calculation

- Calculate ETA based on distance and speed
- Update ETA dynamically
- Display ETA information to users
- Handle traffic simulation factors

8. Notification Services

- Generate notifications for incidents
- Deliver notifications to users
- Display notification history

9. Bilingual Support

- Switch between English and Amharic languages
- Display translated interface components
- Maintain selected language preference

10. Database Operations

- Read and write data correctly
- Maintain data integrity and consistency
- Validate foreign key relationships
- Backup and restore operations

11. Security and Access Control

- Authentication verification
- Authorization based on user roles
- Restricted access to administrative functions
- Secure password storage and validation

12. Error Handling and System Reliability

- Handle invalid user inputs
- Handle missing GPS updates
- Handle database connection failures
- Handle network/server communication errors
- Display appropriate error messages

2.2 Non-Functional Requirements to be Tested

The following non-functional requirements will be tested to ensure that the GizeGo Smart Public Transportation and Real-Time Bus Tracking System meets the required quality standards and provides a reliable user experience.[3]

1. Performance

- Response time for user requests
- Real-time GPS location update performance
- ETA calculation response time
- Database query execution efficiency
- System performance under multiple concurrent users

2. Reliability

- Continuous operation without system failure
- Accurate processing of GPS simulation data
- Consistent delivery of notifications
- Reliable storage and retrieval of data
- Recovery from temporary system failures

3. Availability

- System accessibility during operation
- Availability of backend services
- Availability of database services
- Availability of real-time tracking functionality

4. Security

- Protection of user credentials
- Secure password storage using hashing techniques
- Authentication and authorization enforcement
- Prevention of unauthorized access
- Protection of sensitive user and system data

5. Usability

- Ease of navigation within the mobile application
- User-friendly interface design
- Clarity of system messages and notifications
- Ease of learning and using system features
- Accessibility of major functions by intended users

6. Scalability

- Ability to support an increasing number of users
- Ability to handle additional buses and routes

- Efficient processing of growing GPS and incident data

7. Maintainability

- Ease of updating system components
- Ease of fixing software defects
- Modularity of system architecture
- Readability and maintainability of source code

8. Compatibility

- Compatibility with Android mobile devices
- Compatibility with different screen sizes and resolutions
- Compatibility between the mobile application, backend server, and PostgreSQL database

9. Data Integrity

- Accuracy and consistency of stored data
- Enforcement of database constraints and relationships
- Prevention of duplicate or invalid records
- Preservation of data correctness during updates and deletions

10. Localization Support

- Correct display of English and Amharic languages
- Proper switching between supported languages
- Consistent presentation of translated content throughout the application

2.3 Features Not to Be Tested

The following features are not included in the testing scope of the current version of the GizeGo Smart Public Transportation and Real-Time Bus Tracking System. These features are either outside the scope of the project, planned for future releases, or not implemented as part of the current system requirements.

1. Real GPS Hardware Integration

The current version of the system uses a GPS simulation engine to generate bus location data. Integration with actual GPS tracking devices installed on buses will not be tested because it is not included in this project release.

Reason: Not included in the current version of the system.

2. Online Ticket Purchasing and Payment Processing

The system does not provide functionality for purchasing tickets or processing electronic payments through payment gateways.

Reason: This functionality is outside the project scope and is not implemented in the current release.

3. SMS Notification Services

The system provides in-application notifications only. SMS-based notification delivery is not implemented and therefore will not be tested.

Reason: Not included in the current release due to infrastructure and cost limitations.

4. Integration with External Transportation Systems

Integration with external public transportation management systems, government transportation databases, or third-party transit services is not supported in the current version.

Reason: Outside the scope of the project and not implemented.

5. iOS Device Compatibility

Testing on iOS devices will not be performed. The application is developed and evaluated primarily for Android-based mobile devices.

Reason: Limited project resources and deployment scope.

6. High-Volume Production Load Testing

Large-scale stress testing involving thousands of concurrent users and hundreds of active buses will not be conducted.

Reason: Resource constraints and the academic nature of the project environment.

7. Offline Mode Operation

The system requires an active network connection to access real-time tracking, ETA updates, and backend services. Offline functionality is not implemented.

Reason: Not included in the current system requirements.

8. Automatic Database Failover and Disaster Recovery

Advanced database replication, failover mechanisms, and disaster recovery procedures are not implemented in the current version.

Reason: Considered beyond the scope of the project and intended for future enhancements.

3. Pass/Fail Criteria

The pass/fail criteria define the conditions used to determine whether a test case has been successfully completed or has failed during the testing process. Each test case will be evaluated by comparing the actual system behavior with the expected results specified in the test case.

The success/fail criteria are as follows:

- If the actual output matches the expected result specified in the test case, the test case shall be considered **Passed (Success)**.
- If the actual output differs from the expected result specified in the test case, the test case shall be considered **Failed**.
- A test case shall be marked as **Passed** only when all expected functions, validations, and outputs are executed correctly without producing errors or unexpected behavior.

- A test case shall be marked as **Failed** if the system produces incorrect results, generates unexpected errors, fails to complete the requested operation, or violates the specified requirements.
- A subsystem shall be considered successfully tested when all associated critical and major test cases have passed.
- The overall system shall be considered acceptable for deployment when all critical functionalities, including user authentication, route management, bus tracking, ETA calculation, incident reporting, notifications, and role-based access control, achieve a passing status and no unresolved critical defects remain.

The pass/fail evaluation results will be recorded for each test case and summarized in the final testing report.

4. Approach/Strategy

The testing approach adopted for the GizeGo Smart Public Transportation and Real-Time Bus Tracking System is based on a combination of functional testing, integration testing, system testing, security testing, and user acceptance testing. This approach was selected to ensure that all system components operate correctly both individually and when integrated as a complete system.

Testing will be conducted incrementally throughout the development process. Individual modules will first be tested independently to verify their functionality. After successful module testing, integration testing will be performed to verify communication between the mobile application, backend services, real-time communication services, and PostgreSQL database. Finally, system testing and user acceptance testing will be conducted to validate the complete system against the specified requirements.

4.1 Testing Tools

The following tools will be used during testing:

- **Swagger UI** was used as the primary API testing and documentation tool to validate backend services and verify API functionality.

- **ExpoGO ON Physical Android Devices** for mobile application testing.
- **PostgreSQL** tools for database validation and data integrity verification.
- **Socket.io Testing Utilities** for validating real-time communication and bus location updates.
- **Visual Studio Code Debugger** for debugging and troubleshooting application components.

4.2 Training Requirements

The selected testing tools are widely used and require minimal specialized training. Team members are familiar with the development environment and will utilize existing knowledge of Swagger, PostgreSQL, and Android testing tools during the testing process.

4.3 Metrics to be Collected

The following testing metrics will be collected:

- Number of test cases executed.
- Number of test cases passed.
- Number of test cases failed.
- Defect count by subsystem.
- Defect severity level.
- System response time.
- API response success rate.
- ETA calculation accuracy.
- Real-time GPS update success rate.

4.4 Metric Collection Levels

Metrics will be collected at the following levels:

- **Module Level:** Individual subsystem testing.
- **Integration Level:** Communication between subsystems.
- **System Level:** Complete end-to-end system testing.

- **User Acceptance Level:** Validation of user requirements.

4.5 Configuration Management

Configuration management will be maintained through version control practices and controlled software releases. All changes to source code, database schemas, and configuration files will be documented and tracked throughout the development and testing process. Only approved versions of the software will be used during formal testing activities.

4.6 Test Configurations

Testing will be performed using the following configurations:

Hardware Configuration

- Android smartphone devices.
- Personal computers used for backend hosting and database management.
- Local network and internet connectivity for communication testing.

Software Configuration

- Android Operating System.
- React Native mobile application.
- Node.js and Express.js backend services.
- PostgreSQL database server.
- Socket.io real-time communication framework.

Hardware and Software Combination Testing

Testing will verify the interaction between:

- Android devices and backend services.
- Backend services and PostgreSQL database.
- Mobile application and real-time tracking services.
- Notification services and user interfaces.

4.7 Test Environment

Component	Specification
Client Device	Android Smartphone
Operating System	Android 11+
Frontend	React Native
Backend	Node.js + Express.js
Database	PostgreSQL
Real-Time Communication	Socket.io
API Testing Tool	Swagger UI
Development Environment	Visual Studio Code

4.8 Regression Testing

Regression testing will be conducted whenever modifications are made to existing functionality. The purpose of regression testing is to ensure that newly implemented features or bug fixes do not introduce defects into previously tested components.

Regression testing will focus on:

- Authentication and authorization.

- Route management.
- Bus management.
- GPS simulation.
- ETA calculation.
- Incident management.
- Notification services.

The extent of regression testing will depend on the severity and impact of the identified defects.

4.9 Handling Untestable or Ambiguous Requirements

Any requirement that is unclear, incomplete, inconsistent, or not testable will be reviewed by the project team. Such requirements will be clarified, refined, or corrected before testing activities begin. If a requirement cannot be objectively verified, it will be revised to include measurable acceptance criteria.

4.10 Testing Scope and Coverage

Testing will be performed on the complete integrated system. Special attention will be given to features that involve interactions among multiple subsystems, including real-time bus tracking, ETA prediction, incident reporting, and notification delivery. These components will be tested both individually and as part of the complete operational workflow.

The testing process aims to achieve comprehensive coverage of all functional requirements, security requirements, and major non-functional requirements specified for the GizeGo system.

5. Test cases with specifications

A test case is a set of input data and expected results that exercises a component with the purpose of causing failures and detecting faults [1].

5.1 Functional Test case

Functional testing was conducted to verify that the GizeGo Smart Public Transportation and Real-Time Bus Tracking System performs all specified functional requirements correctly. The

purpose of functional testing is to ensure that each feature of the system operates according to its intended behavior and produces the expected results when provided with valid and invalid inputs.

Table 5.1.1: Test Case Specification for User Login

Name: User Login(for passenger optional for feedback ,Admin and driver)			
Purpose: To verify that registered users can successfully log in to the system using valid email credentials and that invalid login attempts are handled appropriately.			
Test Data: ● Email Address (valid, invalid, empty) ● Password (valid, invalid, empty)			
Input Data	**Expected Result**	**Actual Output**	**Pass/Fail**
Empty email and valid password	"Email cannot be empty"	"Email cannot be empty"	Pass
Valid email and empty password	"Password cannot be empty"	"Password cannot be empty"	Pass
Empty email and empty password	"Required fields cannot be empty"	"Required fields cannot be empty"	Pass
Invalid email format and valid password	"Invalid email format"	"Invalid email format"	Pass
Unregistered email and valid password	"User does not exist"	"User does not exist"	Pass
Valid email and	"Invalid credentials"	"Invalid credentials"	Pass

invalid password			
Valid email and valid password	User is authenticated and redirected to the appropriate dashboard	User is authenticated and redirected to the appropriate dashboard	Pass

Table 5.1.2: Test Case Specification for Route Management

Name: Route Management			
Purpose: To verify that administrators can manage bus routes.			
Test Data: ● Route Name (valid, invalid, empty) ● Start Point (valid, invalid, empty) ● Destination Point (valid, invalid, empty) ● Route ID (valid, invalid)			
Input Data	**Expected Result**	**Actual Output**	**Pass/Fail**
Empty route name	"Route name cannot be empty"	"Route name cannot be empty"	Pass
Empty start point	"Start point cannot be empty"	"Start point cannot be empty"	Pass
Empty destination point	"Destination point cannot be empty"	"Destination cannot be empty"	Pass
Valid route information	"Route added successfully"	"Route added successfully"	Pass

Update existing route	“Route updated successfully”	“Route updated successfully”	Pass
Delete existing route	“Route deleted successfully”	“Route deleted successfully”	Pass

Table 5.1.3: Test Case Specification for Bus Management

Name: Bus Management			
Purpose: To verify that administrators can manage bus information			
Test Data: ● Plate Number (valid, duplicate, empty) ● Bus Status (active, inactive) ● Assigned Route (valid, invalid, empty)			
Input Data	**Expected Result**	**Actual Output**	**Pass/Fail**
Empty plate number	“Plate number cannot be empty”	“Plate number cannot be empty”	Pass
Duplicate plate number	“Plate number already exists”	“Plate number already exists”	Pass
Valid bus information	**“**Bus added successfully”	**“**Bus added successfully”	Pass
Update existing bus	**“**Bus information updated successfully”	**“**Bus information updated successfully”	Pass
Delete existing bus	**“**Bus deleted successfully”	**“**Bus deleted successfully”	Pass

Table 5.1.4: Test Case Specification for ETA Calculation

Name:ETA Calculation			
Purpose: To verify that estimated arrival times are calculated correctly.			
Test Data: ● Valid bus location and route data ● Missing location data ● Missing route information ● Real-time location updates			
Input Data	**Expected Result**	**Actual Output**	**Pass/Fail**
Valid bus location and route data	ETA calculated successfully	ETA calculated successfully	Pass
Missing location data	Unable to calculate ETA	Unable to calculate ETA	Pass
Missing route information	"Route information required"	"Route information required"	Pass
Real- time location update	ETA updated dynamically	ETA updated dynamically	Pass

Table 5.1.5: Test Case Specification for Notification Service

Name: Notification Service			
Purpose: To verify that notifications are generated and delivered correctly			
Input Data	**Expected Result**	**Actual Output**	**Pass/Fail**

New incident (delayed) reported	Notification generated successfully	Notification generated successfully	Pass
Valid notification data	Notification displayed to users	Notification displayed to users	Pass
View notification history	Notification history displayed	Notification history displayed	Pass

Table 5.1.6: Test Case Specification for Role-Based Access Control

Name: Role-Based Access Control			
Purpose: To verify that users can access only authorized functions.			
Input Data	**Expected Result**	**Actual Output**	**Pass/Fail**

Passenger attempts route viewing	Access granted	Access granted	Pass
Passenger attempts bus management	Access denied	Access denied	Pass
Driver attempts bus management	Access denied	Access denied	Pass
Administrator accesses bus management	Access granted	Access granted	Pass
Administrator accesses route management	Access granted	Access granted	Pass

Table 5.1.7: Test Case Specification for Language Switching

Name: Language Switching			
Purpose: To verify that the application supports multilingual functionality.			
Input Data	**Expected Result**	**Actual Output**	**Pass/Fail**

Select English language	Interface displayed in English	Interface displayed in English	Pass
Select Amharic language	Interface displayed in Amharic	Interface displayed in Amharic	Pass
Switch language during operation	Interface updates successfully	Interface updates successfully	Pass

5.2 Non-Functional Test Cases

Non-functional testing was conducted to evaluate the quality attributes of the GizeGo system, including performance, security, reliability, usability, compatibility, and data integrity. These tests were performed to ensure that the system not only satisfies functional requirements but also meets the required standards of efficiency, stability, and security under normal operating conditions.

Table 5.2.1: Performance Testing

Name: System Performance Test
Purpose: To verify that the system responds within an acceptable time under normal operating

conditions.			
Test Scenario	**Expected Result**	**Actual Result**	**Pass/Fail**
User login request	Response time less than 3 seconds	Response time less than 3 seconds	Pass
Route retrieval request	Response time less than 3 seconds	Response time less than 3 seconds	Pass
ETA calculation	ETA generated within 5 seconds	ETA generated within 5 seconds	Pass
GPS location update	Location updated within 5 seconds	Location updated within 5 seconds	Pass

Table 5.2.2: Security Testing

Name: Security Test

Purpose: To verify that unauthorized users cannot access protected resources.			
Test Scenario	**Expected Result**	**Actual Result**	**Pass/Fail**
Access protected API without login	Access denied	Access denied	Pass
Passenger attempts administrator operation	Access denied	Access denied	Pass
Invalid login credentials	Authentication failed	Authentication failed	Pass
Password stored in database	Password is encrypted/hashed	Password is encrypted/hashed	Pass

Table 5.2.3: Reliability Testing

Name: Reliability Test			
Purpose: To verify stable system operation during continuous use.			
Test Scenario	**Expected Result**	**Actual Result**	**Pass/Fail**
Continuous GPS simulation for 1 hour	No system failure	No system failure	Pass
Repeated ETA calculations	Consistent results produced	Consistent results produced	Pass

Table 5.2.4: Usability Testing

Name: Usability Test			
Purpose: To evaluate the ease of use and accessibility of system functions for different user roles.			
Test Scenario	**Expected Result**	**Actual Result**	**Pass/Fail**
Passenger searches and views bus routes	Route information is easy to locate and understand	Route information is easy to locate and understand	Pass
Passenger tracks a bus and views ETA	Bus location and ETA are displayed clearly	Bus location and ETA are displayed clearly	Pass
Driver views assigned route information	Route details are displayed clearly and accurately	Route details are displayed clearly and accurately	Pass
Administrator manages buses and routes	Administrative functions are easy to access and operate	Administrative functions are easy to access and operate	Pass

Table 5.2.5: Compatibility Testing

Name: Compatibility Test			
Purpose: To verify operation on supported Android devices.			
Test Scenario	**Expected Result**	**Actual Result**	**Pass/Fail**

Name: Compatibility Test			
Purpose: To verify operation on supported Android devices.			
Run application on Android device	Application functions correctly	Application functions correctly	Pass
Different screen resolutions	Interface displays correctly	Interface displays correctly	Pass
Backend communication	Data exchanged successfully	Data exchanged successfully	Pass

Table 5.2.6: Data Integrity Testing

Name: Data Integrity Test			
Purpose: To verify consistency and correctness of stored data.			
Test Scenario	**Expected Result**	**Actual Result**	**Pass/Fail**
Create route with valid data	Route stored correctly	Route stored correctly	Pass
Assign bus to route	Foreign key relationships maintained	Foreign key relationships maintained	Pass
Delete route with dependent records	Database constraint enforced	Database constraint enforced	Pass

6. Test Summary Report

Test Category	Total Tests	Passed	Failed
Functional Testing	45	45	0
Security Testing	6	6	0
Performance Testing	4	4	0
Reliability Testing	3	3	0
Usability Testing	7	7	0
Compatibility Testing	3	3	0
Data Integrity Testing	3	3	0
Total	**68**	**68**	**0**

6. Conclusion

The testing activities conducted for the GizeGo Smart Public Transportation and Real-Time Bus Tracking System were aimed at verifying that the system satisfies its specified functional and non-functional requirements. Through a combination of functional, integration, system, security, and usability testing, the major features of the system were thoroughly evaluated.

Functional testing confirmed the correct operation of core system functionalities, including user registration and authentication, bus management, route management, stop management, GPS simulation, real-time bus tracking, ETA calculation, notification services, role-based access control, and bilingual support. The results demonstrated that the system performs according to the specified requirements and produces the expected outputs under both valid and invalid input conditions.[2]

Non-functional testing evaluated important quality attributes such as performance, reliability, security, usability, compatibility, and data integrity. The test results indicated that the system provides acceptable response times, maintains data consistency, enforces access control policies, and delivers a satisfactory user experience across the supported platform.[2]

Based on the completed testing activities and the obtained results, the GizeGo system has been verified to be stable, reliable, and capable of supporting the intended transportation management and real-time bus tracking services. Therefore, the system is considered ready for deployment within the scope and objectives of the project.

References

[1] IEEE Computer Society, *IEEE Standard for Software and System Test Documentation (IEEE Std 829-2008)*, New York, NY, USA: IEEE, 2008.

[2] I. Sommerville, *Software Engineering*, 10th ed., Boston, MA, USA: Pearson, 2016.

[3] International Organization for Standardization (ISO), *ISO/IEC 25010:2011 Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, Geneva, Switzerland: ISO, 2011